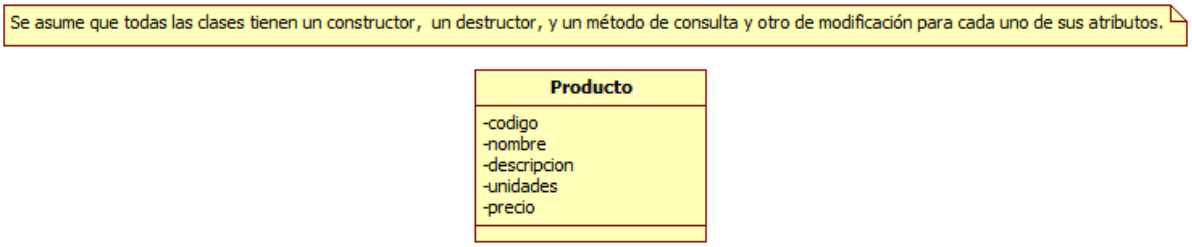
# Ejercicios VI

## DISEÑO DE SOFTWARE

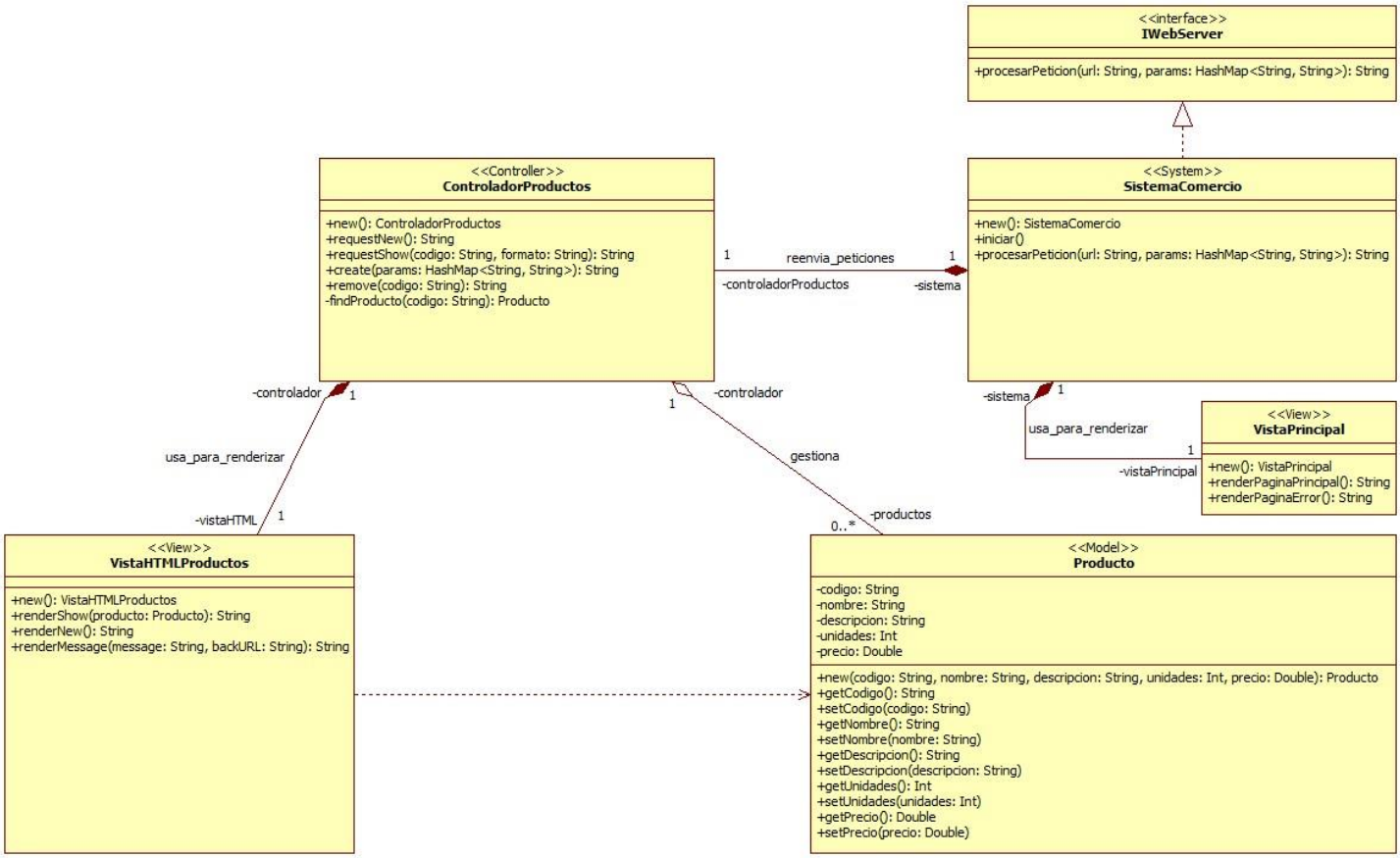
Fundamentos de Ingeniería del Software

# Ejercicio 1

Una empresa de desarrollo de software ha iniciado un nuevo proyecto para crear una plataforma web de comercio electrónico. Esta plataforma debe permitir **dar de alta nuevos productos, listar todos los productos dados de alta, y consultar, editar y eliminar un producto concreto.** La plataforma web debe conocer, para cada producto, su código (que debe ser único y por lo tanto actuará como identificador), su nombre, una descripción, la cantidad de unidades disponibles y su precio en euros. Tras especificar los requisitos de la plataforma web, el equipo de desarrollo de software de la empresa elaboró un **diagrama de clases de análisis** para modelarla. A continuación, se muestra la clase 'Producto' definida en este diagrama.




En la fase de diseño, el equipo de desarrollo de software ha realizado el siguiente **diagrama de clases de diseño** para modelar la plataforma web de comercio electrónico empleando el **patrón arquitectónico MVC (Model-View-Controller o Modelo-Vista-Controlador)**:



Para la resolución del ejercicio, tenga en cuenta la siguiente información:

- Para realizar una petición a la plataforma web se debe llamar al método *procesarPeticion* de *SistemaComercio* indicando la URL solicitada y los parámetros de la petición. Este método devolverá una página web con la respuesta a la petición. La clase *SistemaComercio* atenderá todas las peticiones y redirigirá aquellas que no pueda procesar por si misma a los controladores (en este caso únicamente a *ControladorProductos*). Por ejemplo, si se solicita la URL "/productos/590123" para consultar información sobre el producto con código "590123", *SistemaComercio* llamará al método requestShow("590123","HTML") de *ControladorProductos* para obtener la página web de respuesta.
- Todos los métodos de las clases tipo *View*, *Controller* o *System* que devuelven un String, devuelven el código HTML de una página web que puede ser renderizada por cualquier navegador web. Es decir, realmente estos métodos devuelven páginas web.
- El método "*requestNew*" de *ControladorProductos* devuelve una página web que contiene un formulario donde el usuario final puede especificar la información necesaria para dar de alta un nuevo producto. El controlador obtiene esta página web llamando al método "*renderNew*" de *VistaHTMLProductos*.
- El método "*requestShow*" de *ControladorProductos* devuelve una representación del producto cuyo código se le haya pasado como parámetro en el formato solicitado. En el caso de que este formato sea "HTML", este método devolverá una página web con la información del producto, la cual será obtenida por el controlador llamando al método "*renderShow*" de *VistaHTMLProductos*. En caso de no existir ningún producto con el código indicado, el método "*requestShow*" devolverá una página web con un mensaje de error, la cual será obtenida por el controlador llamando al método "*renderMessage*" de *VistaHTMLProductos*.
- El método "*create*" de *ControladorProductos* crea un nuevo producto a partir de la información recibida en los parámetros. En caso de éxito, devuelve una página web con la información del producto creado (obtenida mediante el método "*requestShow*"). En caso de fallo, devuelve una página web con información sobre el motivo por el cual no se pudo crear el producto (obtenida mediante el método "*renderMessage*" de *VistaHTMLProductos*).

## Apartado 1.1

El diseño propuesto en el enunciado permite dar de alta nuevos productos, así como consultar y eliminar un producto concreto que ha sido previamente dado de alta. Sin embargo, **no permite listar los productos dados de alta ni tampoco editar un producto existente.** Amplíe el **diagrama de clases de diseño** proporcionado en el enunciado para que el diseño de la plataforma web satisfaga estos requisitos.

## Apartado 1.2

Un problema del diseño propuesto en el enunciado es que hay un alto acoplamiento entre la vista (clase *vistaHTMLProductos*) y el modelo (clase *Producto*), lo cual resulta especialmente grave ya que el equipo de desarrollo tomó la decisión de utilizar el patrón arquitectónico MVC precisamente para evitar que esto ocurriera. Modifique el **diagrama de clases de diseño** de la plataforma web para reducir al máximo este acoplamiento aplicando el **principio SOLID de inversión de dependencias**. Asuma que las páginas web que muestran información de los productos muestran su código, nombre, descripción, número de unidades y precio.

## Apartado 1.3

Por último, se desea que la plataforma web permita solicitar información sobre un producto concreto, no solamente en formato HTML, sino también en dos formatos adicionales:

- Formato **Mobile**: la información del producto se proporciona mediante una página web adaptada para dispositivos móviles.
- Formato **PDF**: la información del producto se proporciona mediante un archivo PDF embebido en una página web, el cual podrá ser descargado desde esta página.

Amplíe el **diagrama de clases de diseño** de la plataforma web para que el diseño satisfaga estos requisitos. Asuma que *SistemaComercio* puede obtener el formato solicitado a partir de la URL y que por tanto no es necesaria ninguna modificación en esta clase.

## Apartado 1.4

Imagine que, una vez ya terminada la implementación de la plataforma web de comercio electrónico, se desarrolla una segunda versión de la misma en la cual se debe modificar la implementación de la clase *Producto* debido a la aparición de nuevos requisitos.

¿A qué otras clases podrían afectar estos cambios si la plataforma se hubiera desarrollado de acuerdo al diseño propuesto como solución al apartado 1.3? ¿Y en el caso de que el diseño no se hubiera mejorado reduciendo el acoplamiento entre vistas y controladores?

## Apartado 1.5

El equipo de desarrollo ha agrupado todas las clases del diagrama de clases de diseño anterior en un único componente software llamado *SistemaComercio*. Este equipo también ha creado otro componente llamado *WebClient*, el cual proporciona un cliente web que puede ser utilizado para realizar peticiones a cualquier sistema que implemente la interfaz *IWebServer*.

Realice el **diagrama de componentes** del sistema completo (compuesto por el cliente web y la plataforma web de comercio electrónico) teniendo en cuenta la siguiente información:

- Para que la información sobre los productos se almacene de forma persistente, *SistemaComercio* emplea el sistema gestor de bases de datos (DBMS) MongoDB. Para conectarse a este DBMS, *SistemaComercio* hace uso del componente *MongoDB Java Driver*.
- Para generar los archivos PDF con la información de los diferentes productos, *SistemaComercio* hace uso de una librería llamada *iText*. Esta librería ofrece dos interfaces: iTextCore e iTextPDf2Data. La primera ofrece las funcionalidades básicas y la segunda solamente se utiliza para extraer información de archivos PDF.

## Apartado 1.6

Realice el **diagrama de despliegue** del sistema completo considerando la siguiente información:

- El cliente web se ejecuta en el ordenador personal del usuario final.
- La plataforma web de comercio electrónico (componente *SistemaComercio*) se va a desplegar en un servidor web dedicado.
- El DBMS se va a desplegar en un servidor dedicado diferente al de la plataforma web.
- La comunicación entre el cliente web y la plataforma web se realizará empleando el protocolo "HTTP" y la comunicación entre esta plataforma y el DBMS se realizará empleando el protocolo "MongoDB Wire".

Indique, para cada nodo, los componentes que éste contiene. No es necesario que indique la relación entre los diferentes componentes.

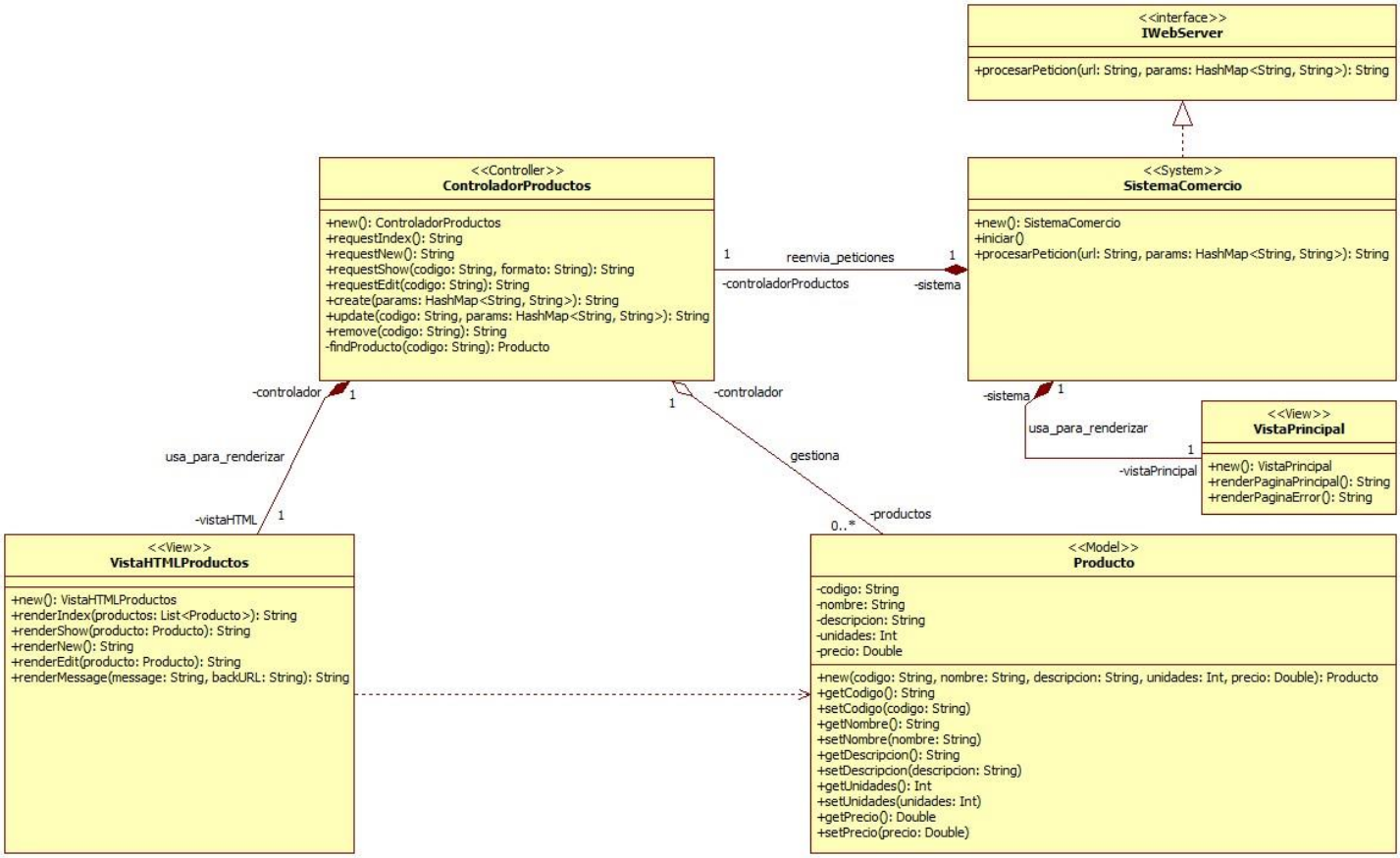
## Apartado 1.7

Para soportar una alta cantidad de peticiones, la empresa ha decidido desplegar un servidor web adicional con una segunda instancia de la plataforma de comercio electrónico e incorporar un balanceador de carga entre los clientes y servidores web. Este balanceador de carga se basa en el uso del servidor HTTP *Apache* y su módulo *ProxyBalancer*. Aunque se ha optado por incorporar dos instancias de la plataforma web, la empresa ha mantenido un único servidor de base de datos.

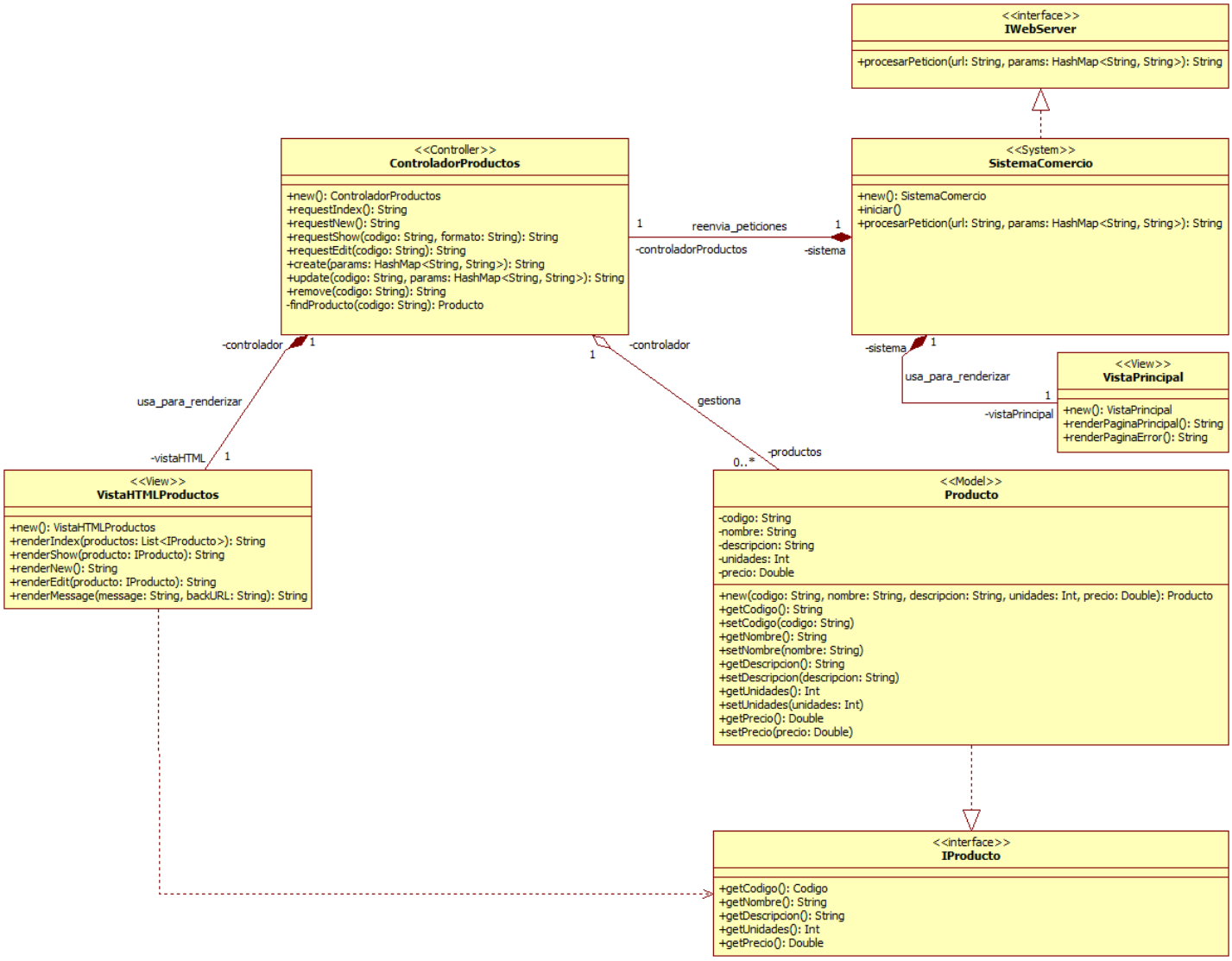
Realice un nuevo **diagrama de despliegue** del sistema completo que refleje los cambios descritos. Indique, para cada nodo, los componentes que éste contiene. No es necesario que indique la relación entre los diferentes componentes.

# Solución

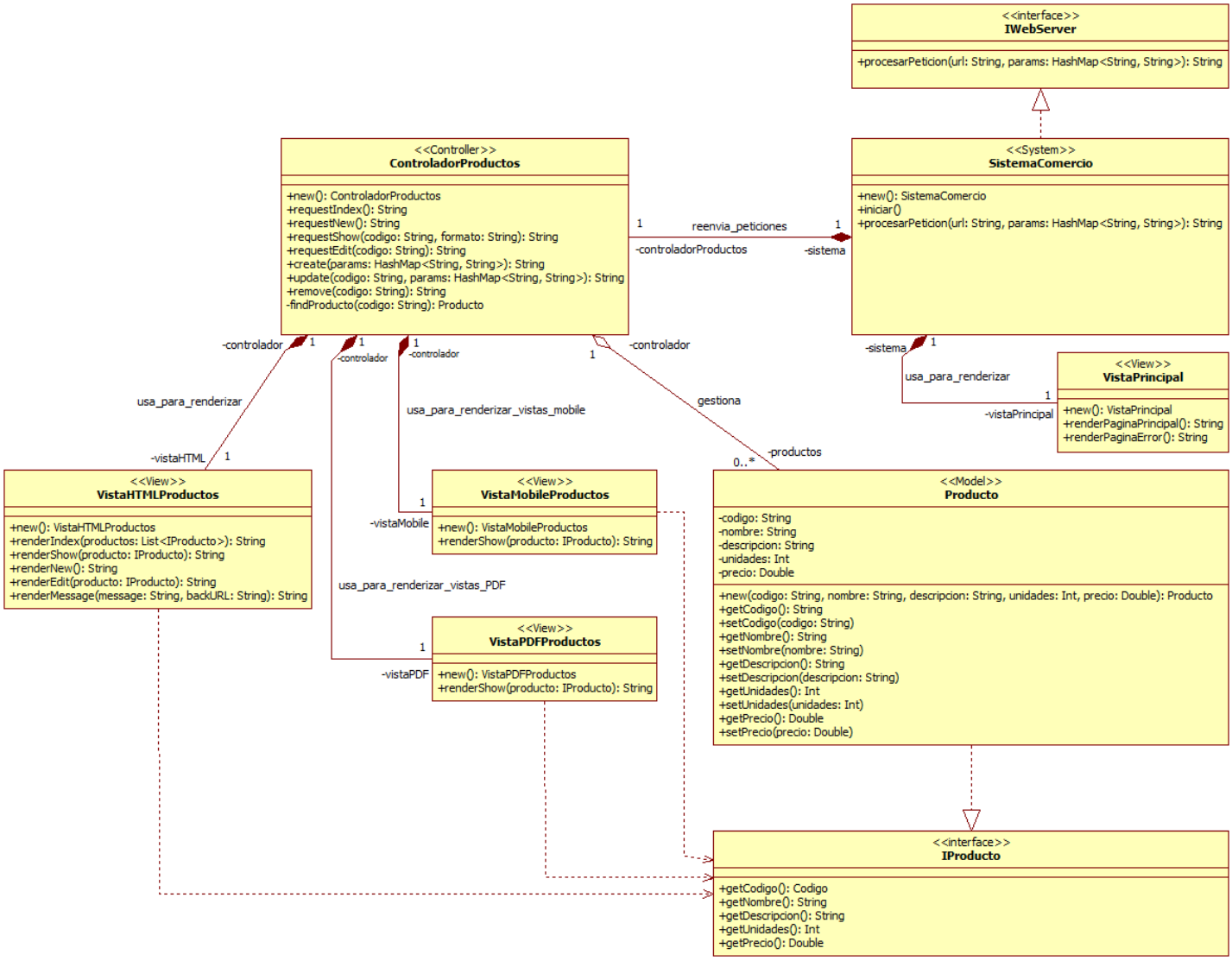
## Apartado 1.1



## Apartado 1.2


<<interface>>
IWebServer
+procesarPeticion(url: String, params: HashMap<String, String>): String
<<Controller>>
ControladorProductos
+new(): ControladorProductos
+requestIndex(): String
+requestNew(): String
+requestShow(codigo: String, formato: String): String
+requestEdit(codigo: String): String
+create(params: HashMap<String, String>): String
+update(codigo: String, params: HashMap<String, String>): String
+remove(codigo: String): String
-findProducto(codigo: String): Producto
<<System>>
SistemaComercio
+new(): SistemaComercio
+iniciar()
+procesarPeticion(url: String, params: HashMap<String, String>): String
1
reenvia_peticiones
1
-controladorProductos
-sistema
-controlador
1
usa_para_renderizar
-vistaHTML
1
1
-controlador
gestiona
0..*
-productos
-sistema
1
usa_para_renderizar
1
-vistaPrincipal
<<View>>
VistaPrincipal
+new(): VistaPrincipal
+renderPaginaPrincipal(): String
+renderPaginaError(): String
<<View>>
VistaHTMLProductos
+new(): VistaHTMLProductos
+renderIndex(productos: List<IProducto>): String
+renderShow(producto: IProducto): String
+renderNew(): String
+renderEdit(producto: IProducto): String
+renderMessage(message: String, backURL: String): String
<<Model>>
Producto
-codigo: String
-nombre: String
-descripcion: String
-unidades: Int
-precio: Double
+new(codigo: String, nombre: String, descripcion: String, unidades: Int, precio: Double): Producto
+getCodigo(): String
+setCodigo(codigo: String)
+getNombre(): String
+setNombre(nombre: String)
+getDescripcion(): String
+setDescripcion(descripcion: String)
+getUnidades(): Int
+setUnidades(unidades: Int)
+getPrecio(): Double
+setPrecio(precio: Double)
<<interface>>
IProducto
+getCodigo(): Codigo
+getNombre(): String
+getDescripcion(): String
+getUnidades(): Int
+getPrecio(): Double

## Apartado 1.3

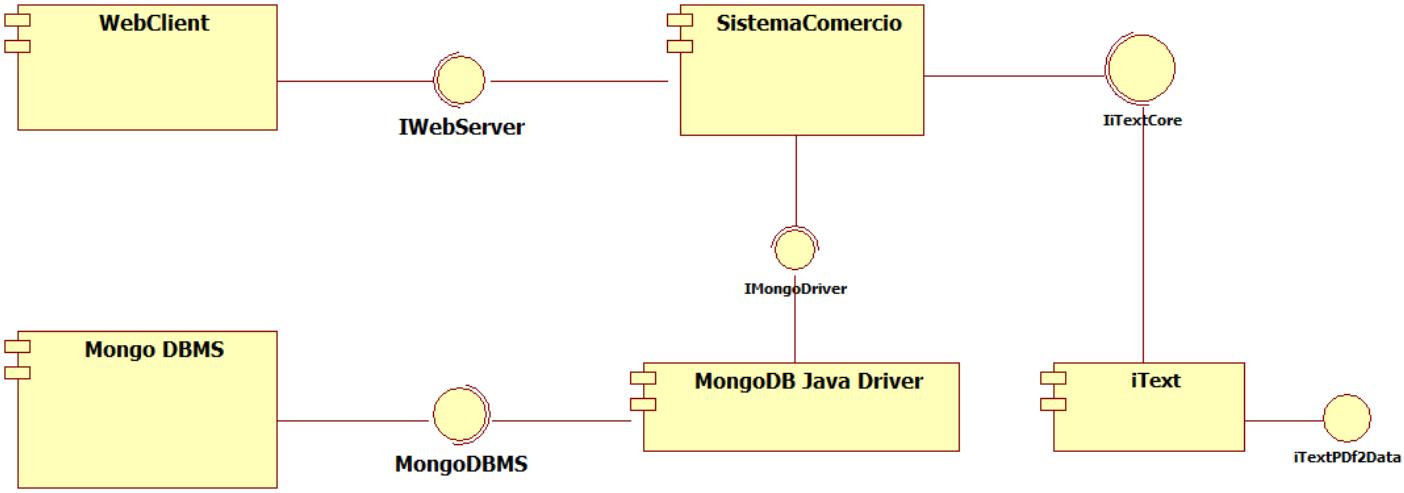



## Apartado 1.4

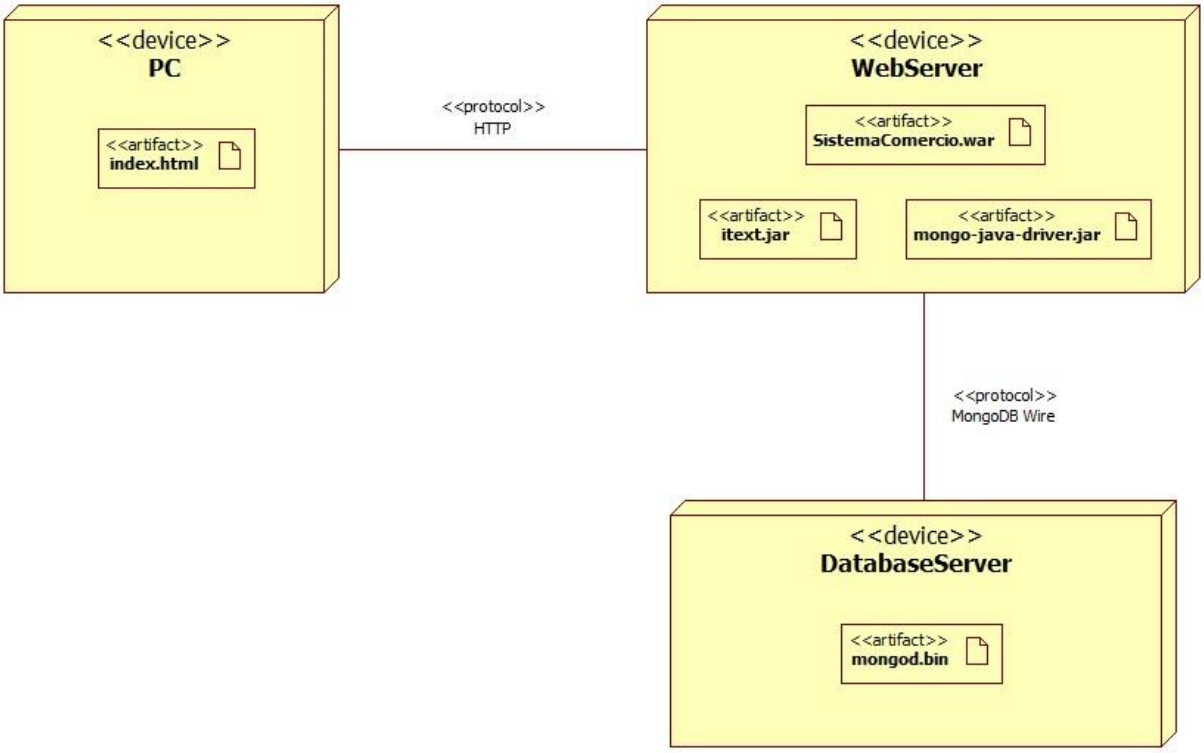
Si la plataforma se hubiera desarrollado de acuerdo al diseño propuesto como solución al apartado 1.3, los cambios realizados sobre la clase *Producto* podrían afectar de forma directa a la clase *ControladorProductos*. Mientras la clase *Producto* siga implementando correctamente la interfaz *IProducto*, sus cambios no afectarán a las vistas (*vistaHTMLProductos*, *vistaMobileProductos* y *vistaPDFProductos*).

En el caso de que el diseño no se hubiera mejorado reduciendo el acoplamiento entre vistas y controladores, un cambio en la clase *Producto* podría afectar directamente tanto a *ControladorProductos* como a todas sus vistas.

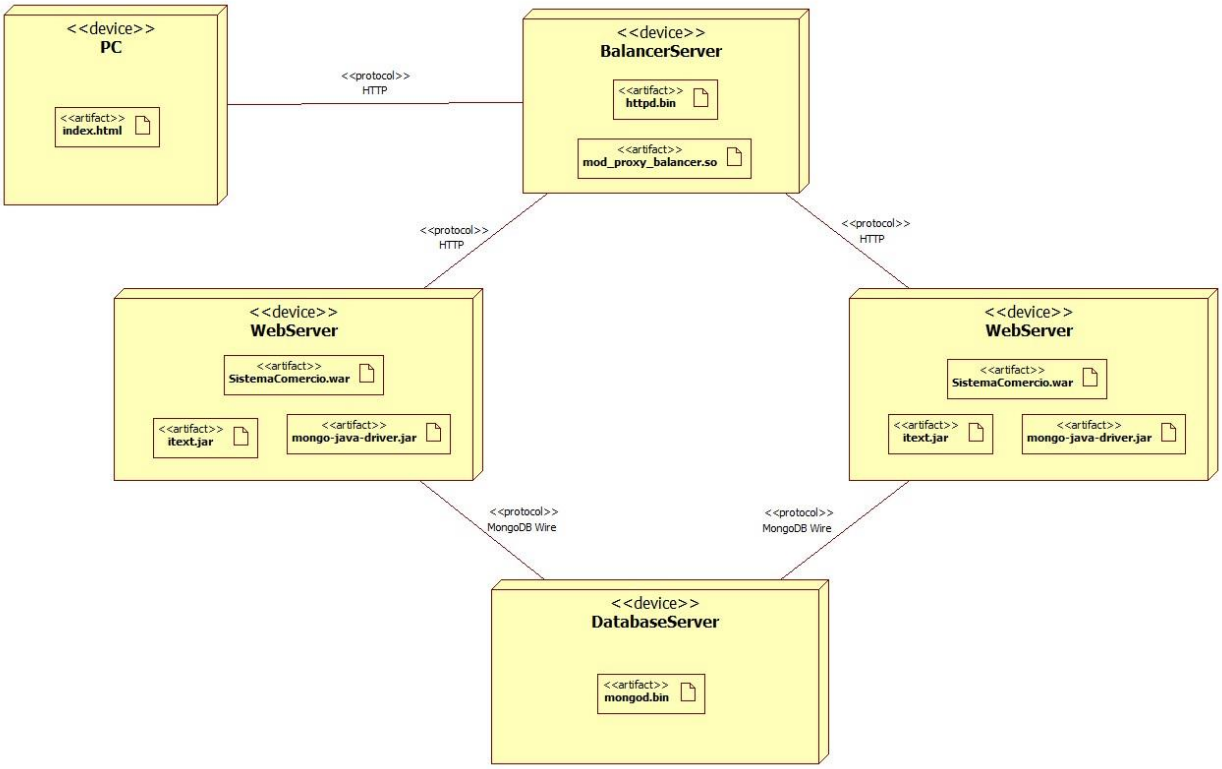
## Apartado 1.5




## Apartado 1.6



## Apartado 1.7


<<device>>
PC
<<artifact>>
index.html
<<protocol>>
HTTP
<<device>>
BalancerServer
<<artifact>>
httpd.bin
<<artifact>>
mod_proxy_balancer.so
<<protocol>>
HTTP
<<protocol>>
HTTP
<<device>>
WebServer
<<artifact>>
SistemaComercio.war
<<artifact>>
itext.jar
<<artifact>>
mongo-java-driver.jar
<<device>>
WebServer
<<artifact>>
SistemaComercio.war
<<artifact>>
itext.jar
<<artifact>>
mongo-java-driver.jar
<<protocol>>
MongoDB Wire
<<protocol>>
MongoDB Wire
<<device>>
DatabaseServer
<<artifact>>
mongod.bin